

FinIA Control | Documentação Principal do Projeto

Arquivo de referência obrigatória. Consulte este documento antes de implementar qualquer funcionalidade, tela ou regra de negócio. Ele define o propósito, escopo, diferenciais e diretrizes do FinIA Control.

Documentos complementares: [docs/REQUISITOS_E_OBJETIVOS.md](#) (requisitos detalhados) e [docs/ESTADO_ATUAL.md](#) (estado técnico atual).

Índice

1. Visão do Produto
2. Público-Alvo
3. Objetivo do Sistema
4. Diferenciais
5. Escopo: pessoa física
6. Onboarding Financeiro
7. Módulos do Sistema
8. Assistente de IA
9. Regras de Negócio
10. Stack Tecnológica
11. Segurança
12. Identidade Visual
13. Tom de Comunicação
14. Ambientes e Deploy
15. Fase Atual do Projeto

1. Visão do Produto

FinIA Control é uma plataforma web responsiva (desktop, tablet e mobile) para controle financeiro de **pessoa física** com inteligência artificial.

Escopo atual (definitivo): apenas **pessoa física**, finanças pessoais ou familiares. Não há modo empresarial, PJ nem gestão para negócios. O cadastro não pergunta "tipo de conta"; todo usuário é tratado como pessoa física (**mode: personal** no banco).

O sistema vai além de anotar gastos: funciona como um **assistente financeiro inteligente** que ajuda o usuário a entender sua vida financeira, prever problemas e tomar melhores decisões com base em dados reais.

Proposta de valor:

- Organizar receitas, despesas, contas fixas, parceladas, dívidas e metas.
- Projetar o futuro financeiro com rendas temporárias e variáveis.

- Oferecer análises, alertas e recomendações via IA.

2. Público-Alvo

Segmento	Necessidade
Pessoas físicas	Organizar vida financeira, controlar gastos, quitar dívidas, atingir metas
Usuários com renda mista	Quem tem salário fixo + renda variável ou extra com prazo definido
Quem busca clareza financeira	Usuários que querem ir além de planilhas e apps básicos de controle

3. Objetivo do Sistema

Ajudar usuários a:

- Organizar a vida financeira de forma clara e centralizada.
 - Controlar **receitas, despesas, contas fixas, contas parceladas, dívidas e metas financeiras**.
 - Receber **análises inteligentes com IA** baseadas nos dados reais cadastrados.
 - Antecipar problemas financeiros e simular impactos de decisões futuras.
 - Tomar decisões mais conscientes sobre gastos, economia e quitação de dívidas.
-

4. Diferenciais

4.1 Assistente financeiro, não apenas controle de gastos

O FinIA Control não se limita a registrar entradas e saídas. Ele interpreta os dados, gera diagnósticos, alertas e planos de ação.

4.2 Rendas variáveis e extras com prazo

Diferencial central: controle de rendas que não são fixas e têm data de término.

Exemplo de cadastro:

"Recebo R\$ 500,00 todo dia 10 até dezembro."

Comportamento esperado:

- A renda entra nas **projeções futuras** até a data de fim.
- O sistema **avisa quando a renda estiver perto de acabar**.
- A IA pode responder perguntas como: *"O que acontece quando minha renda extra acabar?"*

4.3 IA baseada em dados reais

A IA analisa **somente** os dados disponíveis do usuário. **Nunca inventa valores ou informações financeiras.**

4.4 Projeção financeira realista

A projeção de saldo respeita o que **termina** ao longo do tempo: rendas temporárias param na data de fim, despesas parceladas valem só dentro da janela de parcelas, e dívidas deixam de contar após a quitação.

4.5 Foco em finanças pessoais

O produto é dedicado ao controle financeiro pessoal, com uma experiência enxuta e sem a complexidade de gestão empresarial.

5. Escopo: pessoa física

O que é	O que não é
Controle de renda e gastos de pessoa física	Contabilidade ou fluxo de caixa empresarial
Finanças individuais ou do núcleo familiar	Múltiplos modos (pessoal vs. empresa) no cadastro
Um único perfil por usuário no banco (personal)	CNPJ, nota fiscal, centro de custo, etc.

Decisão de produto: o antigo fluxo com escolha de "modo de uso" e wizard longo no início foi **descontinuado**. A experiência atual é enxuta: cadastro -> onboarding curto -> app.

6. Onboarding Financeiro

Fluxo **obrigatório** após o cadastro, em **uma única tela** (não é mais um wizard de vários passos).

Dados coletados hoje no onboarding

Campo	Obrigatório	Onde fica no banco
Profissão / atividade	Não	<code>profiles.profession</code>
Renda fixa mensal	Não	<code>profiles.fixed_monthly_income</code>

Ao enviar o formulário (com estado visível de "**Salvando...**"), o sistema marca `users.onboarding_complete = true` e redireciona para `/dashboard`.

O que o usuário cadastra depois (nos módulos)

Receitas (incl. variáveis/temporárias), despesas e categorias, dívidas e metas são cadastrados em **Receitas, Despesas, Dívidas e Metas**, não no onboarding inicial.

7. Módulos do Sistema

Módulo	Responsabilidade	Status
Autenticação	Cadastro, login, sessão (e-mail + senha), verificação de e-mail	[OK] Implementado
Recuperação de senha	"Esqueci a senha" -> link por e-mail (1h) -> redefinição	[OK] /esqueci-senha, /redefinir-senha
Onboarding financeiro	Uma tela: profissão + renda fixa	[OK] /onboarding
Dashboard	Visão geral, saldo, projeção, alertas e saúde financeira	[OK] Dados reais (Neon)
Receitas	Tabela paginada + CRUD por modal	[OK]
Despesas	Tabela paginada + CRUD por modal	[OK]
Categorias	Organização de despesas	[em breve] Campo texto; sem UI dedicada
Contas fixas	Despesas mensais recorrentes automáticas	[em breve] Repetição automática pendente
Contas parceladas	Geração e acompanhamento de parcelas	[em breve] Modelo parcial
Dívidas	Controle de obrigações e quitação	[OK] CRUD completo
Metas financeiras	Objetivos, progresso e conclusão ao atingir 100%	[OK]
Emprestei	Empréstimos a pessoas (juros, parcelas, pagamentos)	[OK] Oculto por padrão; liberado por usuário pelo admin
Relatórios	Análises visuais e exportação	[em breve] Placeholder
Investimentos	Acompanhamento de aplicações	[em breve] Placeholder
Assistente IA	Chat, diagnósticos e recomendações	[em breve] Placeholder em /ia
Configurações	Perfil, troca de senha, avatar e logout	[OK] Implementado
Admin	Gestão de clientes, planos, papéis e entitlement Emprestei	[OK] /admin
Planos e assinatura	Monetização e limites por plano	[em breve] Cobrança (Stripe) na fase futura

Todas as rotas têm telas de **erro**, **carregamento** e **404** (raiz e dentro do app).

8. Assistente de IA

8.1 Princípio fundamental

A IA nunca deve inventar dados financeiros. Ela analisa somente os dados disponíveis do usuário.

Se não houver dados suficientes, a IA deve informar isso claramente. Nunca preencher lacunas com valores fictícios.

8.2 Perguntas que a IA deve responder

- Quanto gastei este mês?
- Onde estou gastando mais?
- Como posso economizar?
- Qual dívida devo pagar primeiro?
- Posso fazer uma compra de R\$ 1.000 este mês?
- Quanto consigo guardar por mês?
- O que acontece quando minha renda extra acabar?

8.3 Análises e entregáveis gerados pela IA

- Diagnóstico financeiro · Alertas de gastos · Sugestões de economia
- Plano para quitar dívidas · Projeção de saldo futuro
- Análise de categorias · Simulação de compras futuras

Status: o módulo de IA é hoje um placeholder; o princípio "não inventar dados" já está firmado para guiar a implementação.

9. Regras de Negócio

9.1 Isolamento de dados

- Cada usuário **só pode acessar seus próprios dados** (em toda leitura e escrita).

9.2 Tipos de receita

Fixas, variáveis, extras, temporárias - com **dia do mês** (1-31) e, quando temporárias, **data de fim**.

9.3 Tipos de despesa

Fixas, variáveis, parceladas - com categoria, vencimento e status.

9.4 Contas fixas

- Devem **se repetir automaticamente** nos próximos meses (recorrência automática: [em breve]).

9.5 Contas parceladas

- Devem **gerar parcelas mensais** com valor, vencimento e status (pendente/paga/vencida).

9.6 Projeções com rendas temporárias

- Rendas com data de fim aparecem nas projeções **apenas dentro do período válido**.
- O sistema emite **alertas** quando a renda temporária está próxima do encerramento.
- A projeção desconsidera rendas/parcelas/dívidas já encerradas no mês projetado.

9.7 Metas

- Valor **alvo** $\geq$ **valor atual** e > 0 ; **nome único** por usuário.
- Uma meta só pode ser **concluída** ao atingir 100% (valor atual $\geq$ alvo); pode ser reaberta.

9.8 Validações de entrada

- "Dia do mês" validado entre **1 e 31** no front e no servidor.
- Mutations com tratamento de erro (**try/catch**) e sanitização de valores.

9.9 Empréstei (entitlement)

- Não é recurso de plano: é **liberado individualmente pelo admin** por usuário.
- Oculto no menu, rota bloqueada e ausente da landing para quem não tem liberação.

9.10 IA e integridade dos dados

- Respostas e análises derivam **exclusivamente** dos dados cadastrados; simulações são explicitamente projeções.

10. Stack Tecnológica

Arquitetura **full-stack em Next.js**: interface, regras de negócio, autenticação e persistência no mesmo projeto, com Postgres gerenciado na nuvem. Não há API Django separada.

```
Browser -> Next.js (App Router + Server Actions) -> Neon Postgres
      ^
      |
      | src/proxy.ts (proteção de rotas)
```

10.1 Aplicação (Next.js)

Tecnologia	Uso
Next.js 16	App Router, Server Components, Server Actions, proxy.ts
React 19	Interface de usuário
TypeScript	Tipagem estática fim-a-fim

Tailwind CSS v4	Estilização utilitária
Shadcn UI / Radix	Componentes de interface
Framer Motion	Animações (respeita <code>prefers-reduced-motion</code>)
Lucide React	Ícones
Recharts	Gráficos e visualizações

10.2 Dados e persistência

Tecnologia	Uso
Neon	Postgres gerenciado (região <code>aws-sa-east-1</code>)
PostgreSQL 17	Banco relacional
Drizzle ORM	Schema, queries e migrações (<code>src/lib/db/schema.ts</code>)
<code>@neondatabase/serverless</code>	Driver HTTP para ambiente serverless
<code>drizzle-kit</code>	Geração de migrações (<code>drizzle/</code>)

Tabelas: `users`, `sessions`, `password_reset_tokens`, `email_verification_tokens`, `profiles`, `incomes`, `expense_categories`, `expenses`, `debts`, `goals`, `loans`, `loan_payments`.

Migrações recentes: `0007` recuperação de senha · `0008` índices por `user_id` · `0009` `goals.completed` · `0010` `users.loans_enabled`.

Para aplicar SQL no Neon de forma confiável (o `drizzle-kit migrate` trava com o driver serverless), use `node scripts/apply-sql.mjs drizzle/<arquivo>.sql`.

10.3 Estrutura de código relevante

Caminho	Responsabilidade
<code>src/lib/db/client.ts / schema.ts</code>	Conexão Drizzle + definição das tabelas
<code>src/lib/auth/session.ts</code>	Criar/ler/destruir sessão (cookie assinado)
<code>src/lib/actions/*.ts</code>	Mutations (auth, recuperação de senha, CRUDs, onboarding, perfil, admin)
<code>src/lib/finance/summary.ts</code>	Agregados em SQL para os painéis
<code>src/lib/dashboard.ts</code>	Agregação de dados + projeção do dashboard
<code>src/lib/plans/*</code>	Gating por plano e entitlement (Emprestei)
<code>src/components/app/resource-table.tsx</code>	Tabela paginada com CRUD por modal

10.4 Variáveis de ambiente

Variável	Descrição
<code>DATABASE_URL</code>	Connection string do Neon (preferir endpoint <code>-pooler</code>)
<code>SESSION_SECRET</code>	Segredo forte (obrigatório em produção; assina o cookie de sessão)
<code>NEXT_PUBLIC_APP_URL</code>	Base dos links de e-mail (deve diferir por ambiente)
<code>RESEND_API_KEY</code> / <code>EMAIL_FROM</code>	Envio de e-mail (verificação e recuperação)

10.5 Scripts npm

Script	Descrição
<code>npm run dev</code>	Servidor de desenvolvimento
<code>npm run build</code>	Build de produção
<code>npm run db:generate</code>	Gera migrações a partir do schema
<code>npm run db:studio</code>	UI do Drizzle Studio

10.6 Decisão arquitetural (Next.js vs Django)

O back-end foi consolidado no **próprio Next.js** (um deploy, uma linguagem). Um back-end Django continua possível no futuro se houver necessidade de API compartilhada com outros clientes (app mobile, integrações pesadas).

11. Segurança

Item	Implementação
Hash de senha	scrypt (<code>node:crypto</code>)
Sessão	token opaco em <code>sessions</code> + cookie httpOnly assinado com HMAC (<code>SESSION_SECRET</code>)
Prefixo de cookie	<code>__Host-</code> em produção (Secure/Path/sem Domain)
Recuperação de senha	token de 1h, uso único , anti-enumeração , invalida sessões
Troca de senha	encerra as demais sessões do usuário
Rate-limit	login, cadastro, reenvio e recuperação de senha
Gating de planos	no servidor - conteúdo bloqueado não é renderizado nem enviado ao cliente

Avatares	rota <code>/api/avatar/[userId]</code> exige sessão
Autorização	toda Server Action valida <code>getCurrentUser()</code> ; não depende só do <code>proxy.ts</code>

12. Identidade Visual

12.1 Características

- Layout limpo e premium · tom financeiro profissional · cards arredondados com sombras leves · foco em conversão · design moderno e responsivo.

12.2 Paleta e tema

Elemento	Diretriz
Cor principal	Verde financeiro (<code>#059669</code> claro / <code>#00e676</code> escuro)
App	Suporta tema claro e escuro (preferência do usuário)
Landing	Tema escuro fixo (independente da preferência do usuário)
Cards/Componentes	Bordas suaves, cantos arredondados, hierarquia clara

12.3 Responsividade e acessibilidade

- Adaptada a mobile, tablet, notebook e monitores grandes.
- **Scroll padronizado** em todo o sistema; modais com scroll interno.
- Respeita `prefers-reduced-motion`; erros de formulário com `role="alert"`; tabelas navegáveis por teclado.

12.4 Landing page

Moderna, premium, enxuta e orientada à conversão; comunica transformação financeira, não apenas lista de features.

13. Tom de Comunicação

13.1 Princípios

- Clara, humana e comercial · foco em **transformação financeira** · linguagem acessível · empoderamento do usuário.

13.2 Frases de referência

- *"Pare de apenas anotar gastos. Comece a entender seu dinheiro."*
- *"Organize sua renda fixa, variável e extra com previsões inteligentes."*

- "Saiba o impacto de uma compra antes de fazer."
- "Tenha uma IA financeira olhando seus números todos os dias."

13.3 O que evitar

- Tom técnico demais na landing e no produto.
- Promessas sem base nos dados do usuário.
- Linguagem que reduza o produto a "mais um app de gastos".

14. Ambientes e Deploy

Ambiente	Branch	Domínio	Público
Produção	main	finiacontrol.com.br	clientes
Teste/Staging	staging	finaicontrol.vercel.app	testes

- Deploy via **Vercel**; banco em **Neon**.
- O domínio de teste está conectado ao environment **Preview** (branch `staging`).
- **Pendências de ambiente:** banco Neon separado para staging, `noindex` no domínio de teste e `NEXT_PUBLIC_APP_URL` distinto por ambiente.

15. Fase Atual do Projeto

Item	Status
Documentação principal (este arquivo)	[OK] Atualizada
Landing page (dark fixo, enxuta)	[OK]
Cadastro / login / verificação de e-mail	[OK]
Recuperação de senha	[OK]
Onboarding (1 tela)	[OK]
App shell + proteção de rotas	[OK]
Dashboard com dados reais + projeção correta	[OK]
CRUD Receitas, Despesas, Dívidas, Metas (tabelas paginadas)	[OK]
Emprestei como entitlement do admin	[OK]

Segurança (rate-limit, sessão assinada, gating server-side)	[OK]
Responsividade e padronização de scroll	[OK]
Configurações (perfil + troca de senha + logout)	[OK]
Assistente IA com dados reais	[em breve]
Contas fixas recorrentes, parcelamento completo, relatórios	[em breve]
Planos/assinatura (Stripe), banco de staging isolado	[em breve]
Deploy produção (Vercel + env)	[OK] Configurado

Diretriz para novas funcionalidades

1. Consultar este arquivo (e `docs/REQUISITOS_E_OBJETIVOS.md`) para alinhar com visão, regras e diferenciais.
2. Usar a stack atual: **Server Actions** (escrita), **Server Components** (leitura), **Drizzle** (banco), **`getCurrentUser()`** (autorização).
3. Manter identidade visual, responsividade, padrões de scroll e acessibilidade.
4. Garantir as regras de negócio - especialmente **isolamento de dados e IA só com dados reais**.
5. Após mudanças no schema, rodar `npm run db:generate` e aplicar a migração no Neon.

Última atualização: junho/2026. Stack full-stack Next.js 16 + Neon Postgres + Drizzle.